

ATELIER 3 : ROUTAGE ET GUARDS D'ANGULAR

Objectifs et résultats :

Dans cet atelier, vous allez poursuivre le développement de votre application Angular en intégrant un service d'authentification avec une protection de route utilisant des guards. Vous ajouterez également une page d'accueil (**Home**) qui affiche des informations sur votre boutique en ligne, ainsi qu'un composant de connexion (**Signin**) pour permettre aux utilisateurs de se connecter à l'application. Vous allez également ajouter une constante nommée **PRODUCTS** qui contient la liste des produits, et implémenter la fonctionnalité permettant de supprimer un produit de la liste.

1. Création de la page d'accueil (Home) :

- Créez un composant Angular nommé **HomeComponent**.
- Implémentez la logique nécessaire pour afficher des informations sur votre boutique en ligne, telles qu'une description, des promotions, etc.

2. Ajout du composant de connexion (Signin) :

- Créez un composant Angular nommé **SigninComponent**.
- Ce composant affiche deux boutons : **Signin** et **Signout**.

3. Configuration du module de routage :

- Configurez le routage dans ce module pour gérer la navigation entre les différentes pages de votre application.
- Ajoutez une barre de navigation en haut de votre application.
- Incluez des liens pour naviguer entre toutes les pages.

4. Mise en place du service d'authentification :

- Créez un service Angular nommé **AuthService** pour gérer l'authentification des utilisateurs.
- Implémentez deux méthodes dans ce service : **signin** et **signout**, qui simulent respectivement la connexion et la déconnexion de l'utilisateur.
- Lorsque l'utilisateur clique sur le bouton **Signin**, appelez la méthode **signin** du service d'authentification pour simuler la connexion.
- Lorsque l'utilisateur clique sur le bouton **Signout**, appelez la méthode **signout** du service d'authentification pour simuler la déconnexion.

5. Protection des routes avec des guards :

- Créer un guard **AuthGuard** pour empêcher l'accès aux pages protégées aux utilisateurs non authentifiés (Pour le moment uniquement la page de gestion de produits nécessite une authentification).

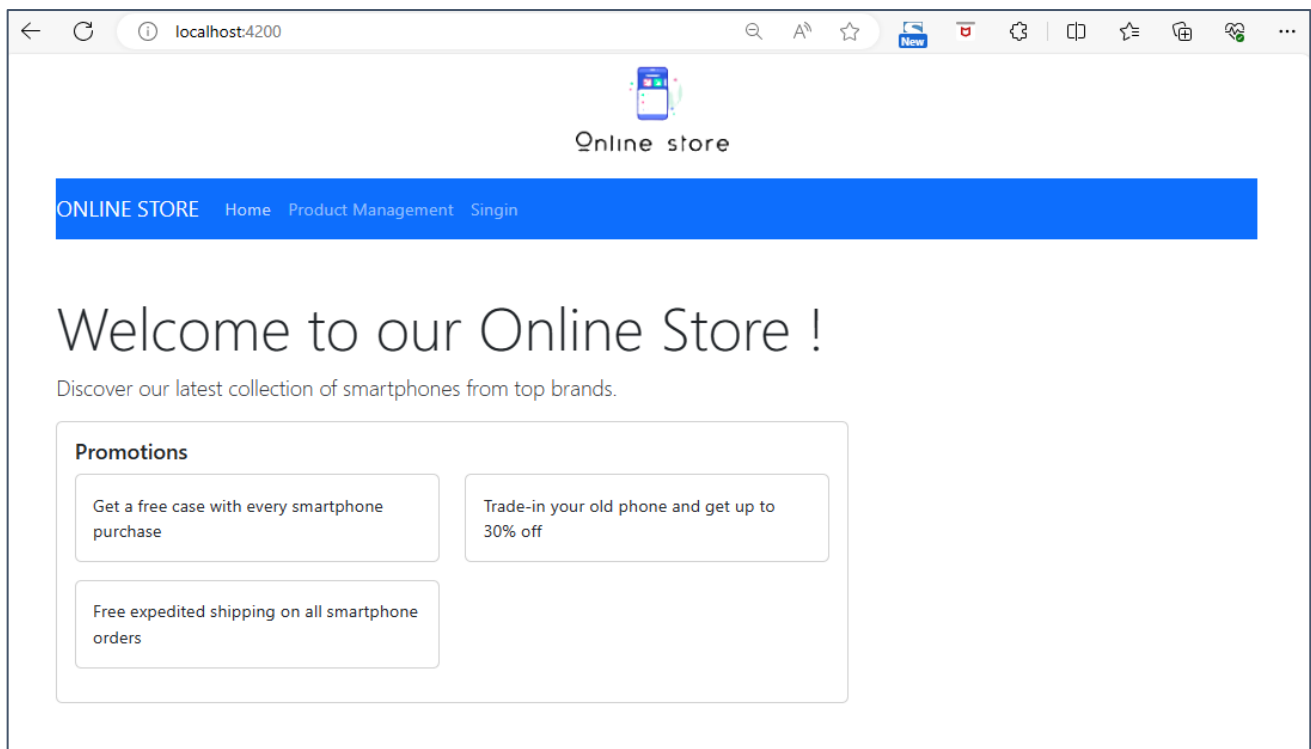
6. Ajout de la constante PRODUCTS :

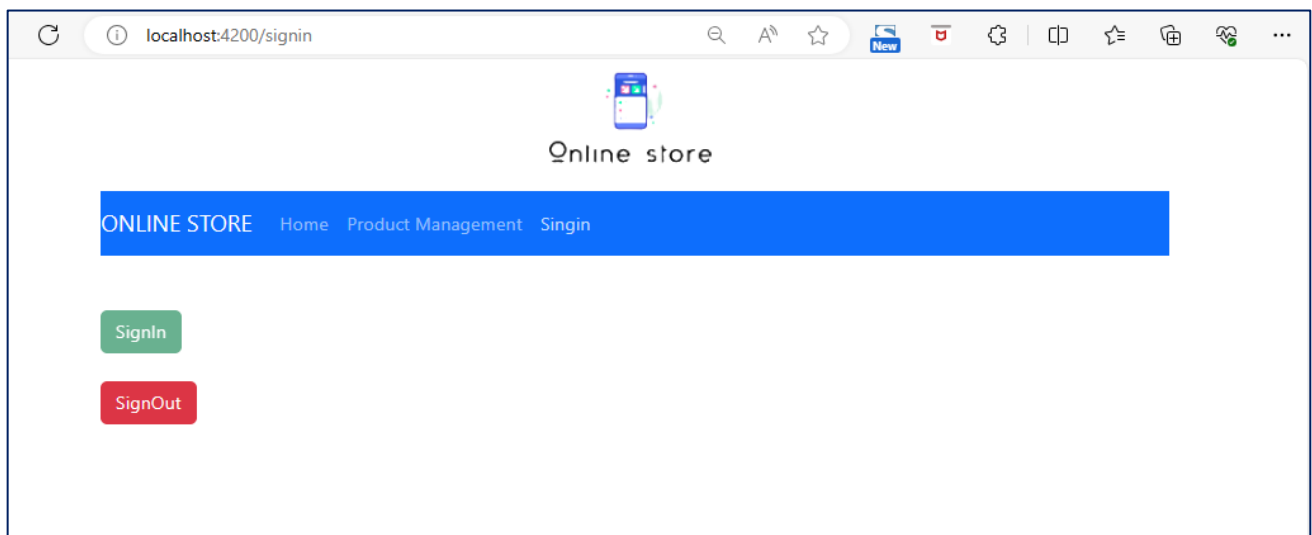
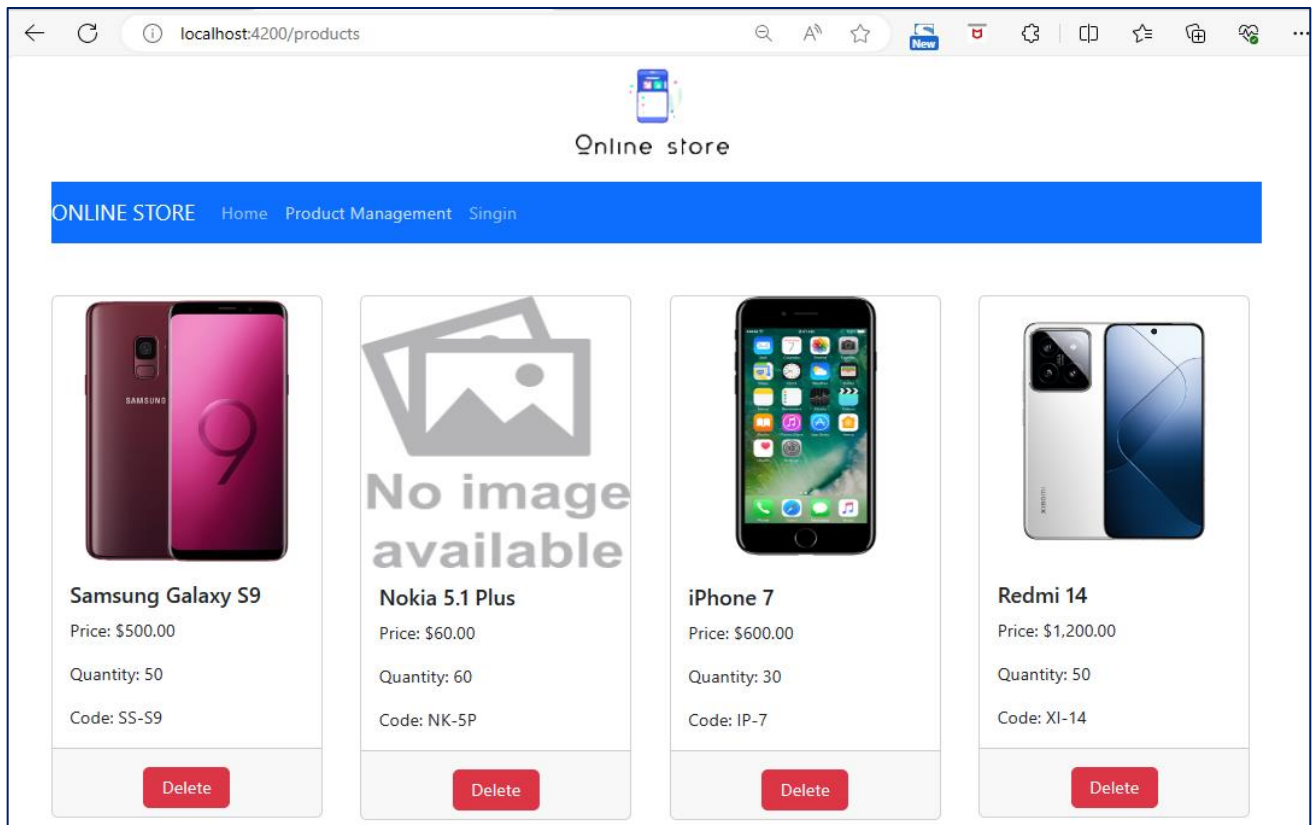
- Créez un fichier de constantes (par exemple **constants.ts**) pour stocker vos constantes.
- Ajoutez une constante nommée **PRODUCTS** qui contient la liste statique des produits.
- N'oubliez pas de mettre à jour le **ProductService** pour inclure cette liste de produits afin qu'elle puisse être utilisée dans votre application.

7. Implémentation de la fonctionnalité de suppression de produit :

- Ajoutez un bouton de suppression à chaque produit dans la liste.
- Implémentez la logique nécessaire pour supprimer un produit de la liste lorsque le bouton est cliqué : Ajouter la méthode **deleteProduct(id: number)** pour inclure la logique de suppression d'un produit.
- Assurez-vous que la logique est implémentée au niveau du composant et du service.

Voici un exemple de ce que vous pouvez avoir comme résultat d'affichage :





Bon Travail